

DAY 2 – BUILDING A SIMPLE PORT SCANNER USING PYTHON

The purpose of the project was to understand the basic principle behind port scanning by implementing a small scanner myself rather than directly using an established tool such as Nmap.

The scanner was tested against the gateway IP address of my own home network.

OBJECTIVES:

The main objectives of this project were:

- Understand what a port scanner does.
- Learn how network ports can be tested.
- Build a simple scanner using Python.
- Accept a target IP address through the command line.
- Display only the ports identified as open.
- Gain practical understanding of basic network reconnaissance.

WHAT I LEARNT :

A network port is a logical endpoint used by applications and services to communicate over a network.

Ports are identified by numbers, ranging from 0 to 65535.

For example, commonly used ports include:

- Port 22 — SSH
- Port 80 — HTTP
- Port 443 — HTTPS

A port scanner checks a computer's network ports to find out which ports are open and what services may be running on them.

Open port → a service is listening/accepting connections.

Closed port → no service is listening.

Filtered port → a firewall or network filter may be blocking the scan.

This tells you that the target has SSH, HTTP, and SMB (etc)-related services exposed.

Network ports can be tested by **attempting to establish a network connection to a specific IP address and port**.

For this Port Scanner, the basic process is:

1. Choose a target IP, e.g. 192.168.1.1.
2. Choose a port, e.g. 80.
3. Try to create a TCP connection to 192.168.1.1:80.
4. If the connection succeeds → **port is open**.
5. If it fails/times out → **port is not reachable/open**.

This project gave me a practical understanding of the first stage of network reconnaissance.

Instead of treating a port scanner as a black-box security tool, I implemented a simple version myself and observed how it could identify accessible ports on a device.

TOOLS AND ENVIRONMENT :

Programming language: Python

Development environment: Python IDLE

Execution environment: Windows Command Prompt

Target: My own home network gateway/router

IMPLEMENTATION:

The scanner was written as a Python .py file using Python IDLE.

The program accepts the target IP address as a command-line argument.

The script was executed from Windows Command Prompt using: `python3 portscanner.py <target-ip>`

For testing, the target IP was the gateway IP address of my own laptop's home network.

Example: `python3 portscanner.py <gateway-ip>`

CODE:

```
#!/bin/python3

import sys
import socket
from datetime import datetime

#defining the target
if len(sys.argv) == 2:
    target = socket.gethostbyname(sys.argv[1])    #translate hostname to IPv4

else:
    print("Invalid amount of arguments")
```

```

print("\n")
print("Scanning target: ", target)
print("Time started :",str(datetime.now()))
print("\n")

try:
    for port in range(50,85):
        s = socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        socket.setdefaulttimeout(1)
        reset = s.connect_ex((target,port))
        if reset == 0:
            print(f"Port {port} is open")
        s.close()          #attempting to scan home router
except KeyboardInterrupt:
    print("\n Exiting program.")
    sys.exit()
except socket.gaierror:
    print("Hostname could not be resolved. ")
    sys.exit()
except socket.error:
    print("Could not connect to server")
    sys.exit()

```

- **Imports:** sys handles command-line arguments, socket handles network connections, and datetime records the scan start time.
- **Gets the target:** sys.argv[1] takes the IP/hostname provided when running the script, while socket.gethostbyname() converts a hostname into an IPv4 address.
- **Displays scan information:** It prints the target IP and the time at which the scan started.
- **Scans ports 50–84:** range(50,85) goes through each port from **50 to 84** and creates a TCP socket for each attempt.
- **Tests each port:** connect_ex((target, port)) attempts a TCP connection. A return value of **0 means the connection succeeded**, so the script prints that the port is open.
- **Closes each connection:** s.close() releases the socket after each port is tested.
- **Handles errors:** KeyboardInterrupt, DNS resolution errors (gaierror), and general socket errors are caught so the program can exit cleanly instead of crashing.

socket → Provides networking functionality; in your code, it creates TCP connections to test whether ports are open.

sys → Provides access to Python's system-level functions; here, it reads **command-line arguments** (sys.argv) and exits the program using sys.exit().

- **socket.AF_INET** → tells Python to use **IPv4 addresses** (e.g. 192.168.1.1).
- **socket.SOCK_STREAM** → tells Python to use a **TCP connection**, which is connection-oriented and reliable.

OUTPUT:

```
Wireless LAN adapter Wi-Fi:

Connection-specific DNS Suffix  . : 
Link-local IPv6 Address . . . . . : fe80::4967:2522:dff3:5511%2
IPv4 Address. . . . . : 192.168.1.3
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.1.1

C:\Users\nehaa\Downloads>python3 portscanner.py 192.168.1.1

Scanning target: 192.168.1.1
Time started : 2026-08-13 21:35:58.118057

Port 53 is open
Port 80 is open
```

INFERENCE:

Port 53 is normally used by DNS (Domain Name System), which translates domain names such as google.com into IP addresses.

Port 80 is the standard port for HTTP, the protocol used for unencrypted web traffic.

Port 80 being open on the router likely meant the router had a **web-based administration interface or another HTTP service** listening there.

An open port does **not automatically mean a vulnerability exists**. It simply means a service is accepting connections on that port!

Router was likely running services that needed those ports!

Further enumeration would be required to determine the service, version, configuration, and potential security issues associated with an open port.

This is one of the areas where more advanced tools such as Nmap provide significantly more information than a basic port scanner.

Nmap can perform much more extensive reconnaissance, including service/version detection, operating-system detection, and scripting-based enumeration.

Building a basic scanner first helped me understand the underlying concept before moving on to more advanced scanning tools.